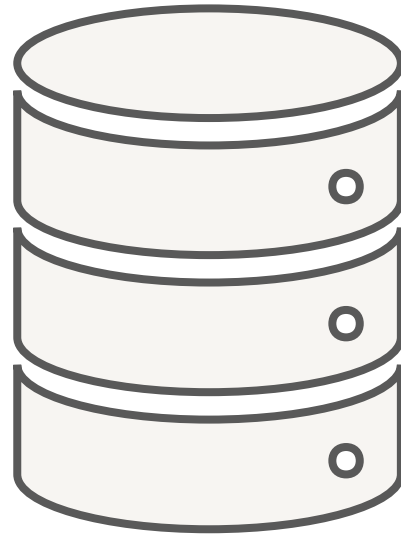

Générateur de requêtes (Query Builder)

Introduction

Le **Query builder** de base de données de Laravel fournit une interface pratique et fluide pour créer et exécuter des requêtes de base de données. Il peut être utilisé pour effectuer la plupart des opérations de base de données dans votre application et fonctionne parfaitement avec tous les systèmes de base de données pris en charge par Laravel.

Le **Query builder** utilise la liaison des paramètres PDO pour protéger votre application contre les attaques par injection SQL. Il n'est pas nécessaire de nettoyer ou d'assainir les chaînes transmises au constructeur de requêtes en tant que liaisons de requêtes.

Exécution de requêtes de base de données



Récupération de toutes les lignes d'une table

Vous pouvez utiliser la méthode **table** fournie par la façade **db** pour commencer une requête. La méthode `table` retourne une instance du générateur de requêtes fluide pour la table donnée, ce qui vous permet de chaîner davantage de contraintes sur la requête, puis de récupérer les résultats de la requête à l'aide de la méthode **get** :

```
public function index()  
{  
    $users = DB::table('users')->get();  
  
    return view('user.index', ['users' => $users]);  
}
```

Récupération de toutes les lignes d'une table

La méthode **get** renvoie une instance **Illuminate\Support\Collection** contenant les résultats de la requête où chaque résultat est une instance de l'objet **PHP stdClass**. Vous pouvez accéder à la valeur de chaque colonne en accédant à la colonne comme une propriété de l'objet :

```
$users = DB::table('users')->get();  
  
foreach ($users as $user) {  
    echo $user->name;  
}
```

Récupération d'une seule ligne/colonne d'un tableau

Si vous n'avez besoin que de récupérer une seule ligne d'une table de base de données, vous pouvez utiliser la méthode **first** de la façade **DB**. Cette méthode renvoie un objet stdClass unique :

```
$user = DB::table('users')->where('name', 'John')->first();  
  
return $user->email;
```

Récupération d'une seule ligne/colonne d'un tableau

Si vous n'avez pas besoin d'une ligne entière, vous pouvez extraire une seule valeur d'un enregistrement en utilisant la méthode **value**. Cette méthode renvoie directement la valeur de la colonne :

```
$email = DB::table('users')->where('name', 'John')->value('email');
```

Pour récupérer une seule ligne par la valeur de sa colonne id, utilisez la méthode **find** :

```
$user = DB::table('users')->find(3);
```

Récupération d'une liste de valeurs de colonnes

Si vous souhaitez récupérer une instance de **Illuminate\Support\Collection** contenant les valeurs d'une seule colonne, vous pouvez utiliser la méthode **pluck**. Dans cet exemple, nous allons récupérer une collection de titres d'utilisateurs :

```
use Illuminate\Support\Facades\DB;

$titles = DB::table('users')->pluck('title');

foreach ($titles as $title) {
    echo $title;
}
```


Récupération d'une liste de valeurs de colonnes

Vous pouvez spécifier la colonne que la collection résultante doit utiliser comme clé en fournissant un second argument à la méthode **pluck** :

```
$titles = DB::table('users')->pluck('title', 'name');  
  
foreach ($titles as $name => $title) {  
    echo $title;  
}
```

Découpage des résultats

Si vous devez travailler avec des milliers d'enregistrements de base de données, envisagez d'utiliser la méthode **chunk** fournie par la façade **DB**. Cette méthode récupère un petit fragment de résultats à la fois et alimente chaque fragment dans une fermeture pour traitement. Par exemple, récupérons l'intégralité de la table `users` par tranches de 100 enregistrements à la fois :

```
DB::table('users')->orderBy('id')->chunk(100, function ($users) {  
    foreach ($users as $user) {  
        //  
    }  
});
```

Découpage des résultats

Vous pouvez empêcher le traitement d'autres morceaux en renvoyant **false** de la fermeture :

```
DB::table('users')->orderBy('id')->chunk(100, function ($users) {  
    // Process the records...  
  
    return false;  
});
```

Résultats en streaming fainéant (Lazy Streaming)

La méthode **lazy()** fonctionne de la même manière que la méthode **chunk** dans le sens où elle exécute la requête par morceaux. Toutefois, au lieu de transmettre chaque fragment dans une fonction de rappel, la méthode **lazy()** renvoie une **LazyCollection**, qui vous permet d'interagir avec les résultats sous la forme d'un flux unique :

```
use Illuminate\Support\Facades\DB;

DB::table('users')->orderBy('id')->lazy()->each(function ($user) {
    //
});
```

Résultats en streaming fainéant (Lazy Streaming)

Une fois encore, si vous prévoyez de mettre à jour les enregistrements récupérés tout en les parcourant, il est préférable d'utiliser les méthodes **lazyById** ou **lazyByIdDesc**. Ces méthodes pagineront automatiquement les résultats en fonction de la clé primaire de l'enregistrement :

```
DB::table('users')->where('active', false)
->lazyById()->each(function ($user) {
    DB::table('users')
        ->where('id', $user->id)
        ->update(['active' => true]);
});
```

Les agrégats

Le générateur de requêtes fournit également une variété de méthodes pour récupérer des valeurs agrégées comme le **nombre**, le **maximum**, le **minimum**, la **moyenne** et la **somme**. Vous pouvez appeler n'importe laquelle de ces méthodes après avoir construit votre requête :

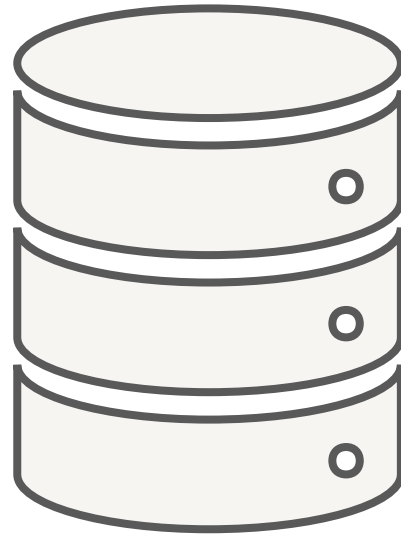
```
use Illuminate\Support\Facades\DB;  
  
$users = DB::table('users')->count();  
  
$price = DB::table('orders')->max('price');
```

Déterminer l'existence des enregistrements

Au lieu d'utiliser la méthode **count** pour déterminer s'il existe des enregistrements qui correspondent aux contraintes de votre requête, vous pouvez utiliser les méthodes **exists** et **doesntExist** :

```
if (DB::table('orders')->where('finalized', 1)->exists()) {  
    // ...  
}  
  
if (DB::table('orders')->where('finalized', 1)->doesntExist()) {  
    // ...  
}
```

Les instructions `SELECT`

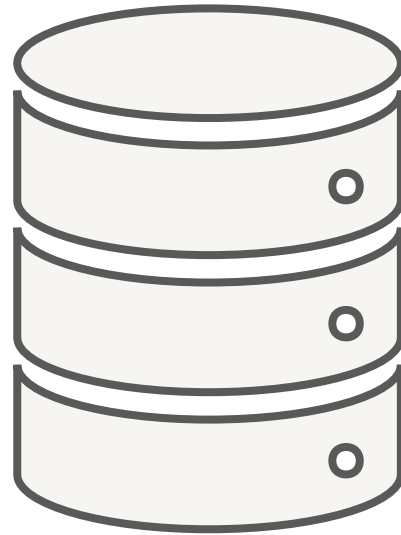


Spécifier une clause SELECT

Vous ne souhaitez pas toujours sélectionner toutes les colonnes d'une table de base de données. En utilisant la méthode **select**, vous pouvez spécifier une clause "select" personnalisée pour la requête :

```
use Illuminate\Support\Facades\DB;  
  
$users = DB::table('users')  
    ->select('name', 'email as user_email')  
    ->get();
```

Expressions brutes



Expression de chaîne brute

Parfois, vous pouvez avoir besoin d'insérer une chaîne arbitraire dans une requête. Pour créer une expression de chaîne brute, vous pouvez utiliser la méthode **raw** fournie par la façade **DB** :

```
$users = DB::table('users')
    ->select(DB::raw('count(*) as user_count, status'))
    ->where('status', '<>', 1)
    ->groupBy('status')
    ->get();
```

Les méthodes brutes

La méthode **selectRaw** peut être utilisée à la place de `addSelect(DB::raw(/* ... */))`. Cette méthode accepte un tableau facultatif de liens comme deuxième argument :

```
$orders = DB::table('orders')  
    ->selectRaw('price * ? as price_with_tax', [1.0825])  
    ->get();
```

Les méthodes brutes

Les méthodes **whereRaw** et **orWhereRaw** peuvent être utilisées pour injecter une clause "**where**" brute dans votre requête. Ces méthodes acceptent un tableau facultatif de liens comme deuxième argument :

```
$orders = DB::table('orders')  
    ->whereRaw('price > IF(state = "TX", ?, 100)', [200])  
    ->get();
```

Les méthodes brutes

Les méthodes **havingRaw** et **orHavingRaw** peuvent être utilisées pour fournir une chaîne brute comme valeur de la clause "**having**". Ces méthodes acceptent un tableau facultatif de liaisons comme deuxième argument :

```
$orders = DB::table('orders')
    ->select('department', DB::raw('SUM(price) as total_sales'))
    ->groupBy('department')
    ->havingRaw('SUM(price) > ?', [2500])
    ->get();
```

Les méthodes brutes

La méthode **orderByRaw** peut être utilisée pour fournir une chaîne brute comme valeur de la clause "**order by**" :

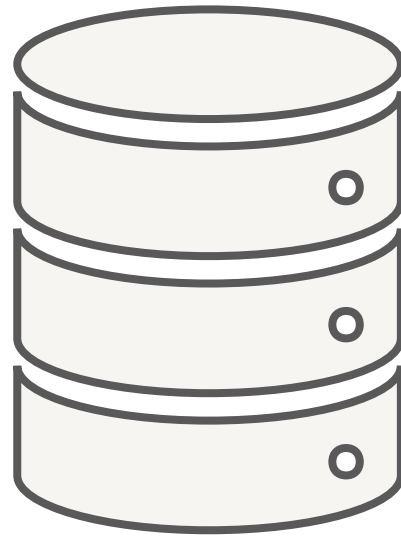
```
$orders = DB::table('orders')  
    ->orderByRaw('updated_at - created_at DESC')  
    ->get();
```

Les méthodes brutes

La méthode **groupByRaw** peut être utilisée pour fournir une chaîne brute comme valeur de la clause **group by** :

```
$orders = DB::table('orders')  
    ->select('city', 'state')  
    ->groupByRaw('city, state')  
    ->get();
```


Joins



Clause de jonction interne

Pour effectuer une « INNER JOIN » de base, vous pouvez utiliser la méthode **join** sur une instance du générateur de requêtes. Le premier argument passé à la méthode **join** est le nom de la table que vous devez joindre, tandis que les autres arguments spécifient les contraintes de colonne pour la jointure. Vous pouvez même joindre plusieurs tables dans une seule requête :

```
$users = DB::table('users')  
    ->join('contacts', 'users.id', '=', 'contacts.user_id')  
    ->join('orders', 'users.id', '=', 'orders.user_id')  
    ->select('users.*', 'contacts.phone', 'orders.price')  
    ->get();
```

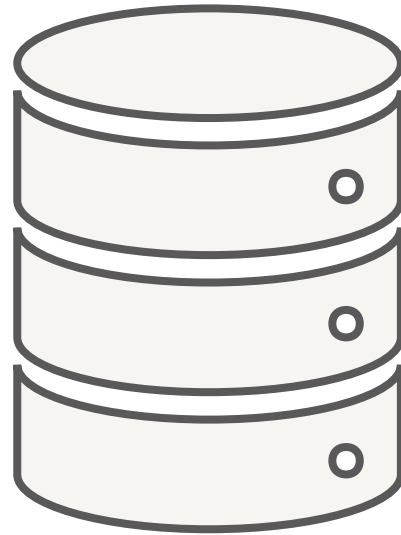
Left Join / Right Join

Si vous souhaitez effectuer une "jointure gauche" ou une "jointure droite" au lieu d'une "jointure interne", utilisez les méthodes **leftJoin** ou **rightJoin**.

```
$users = DB::table('users')  
    ->leftJoin('posts', 'users.id', '=', 'posts.user_id')  
    ->get();
```

```
$users = DB::table('users')  
    ->rightJoin('posts', 'users.id', '=', 'posts.user_id')  
    ->get();
```

Unions



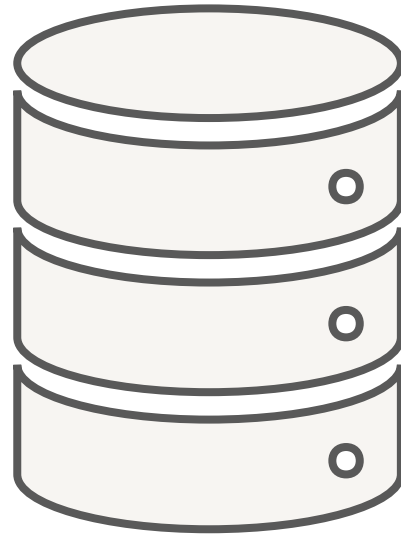
Unions

Le générateur de requêtes fournit également une méthode pratique pour "réunir" deux ou plusieurs requêtes. Par exemple, vous pouvez créer une requête initiale et utiliser la méthode **union** pour l'unir à d'autres requêtes :

```
$first = DB::table('users')
    ->whereNull('first_name');

$users = DB::table('users')
    ->whereNull('last_name')
    ->union($first)
    ->get();
```

Clauses Where de base



La clause Where

Vous pouvez utiliser la méthode **where** du générateur de requêtes pour ajouter des clauses "**where**" à la requête. L'appel le plus basique à la méthode **where** requiert trois arguments.

```
$users = DB::table('users')  
    ->where('votes', '=', 100)  
    ->where('age', '>', 35)  
    ->get();
```

La clause Where

Par commodité, si vous voulez vérifier qu'une colonne est = à une valeur donnée, vous pouvez passer la valeur comme deuxième argument à la méthode **where**. Laravel supposera que vous souhaitez utiliser l'opérateur = :

```
$users = DB::table('users')  
    ->where('votes', 100)  
    ->where('age', '>', 35)  
    ->get();
```


La clause Where

Vous pouvez également passer un **tableau de conditions** à la fonction **where**. Chaque élément du tableau doit être un tableau contenant les trois arguments généralement transmis à la méthode **where** :

```
$users = DB::table('users')->where([  
    ['status', '=', '1'],  
    ['subscribed', '<>', '1'],  
])->get();
```

La clause orWhere

Lorsque vous enchaînez des appels à la méthode **where** du constructeur de requêtes, les clauses "where" seront jointes à l'aide de l'opérateur **and**. Cependant, vous pouvez utiliser la méthode **orWhere** pour joindre une clause à la requête en utilisant l'opérateur **or**.

```
$users = DB::table('users')  
    ->where('votes', '>', 100)  
    ->orWhere('name', 'John')  
    ->get();
```

La clause WhereNot

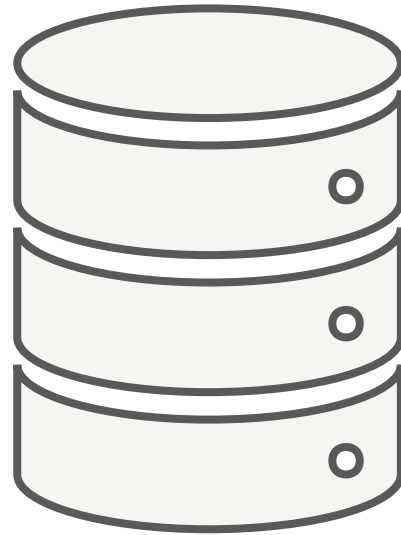
Les méthodes **whereNot** et **orWhereNot** peuvent être utilisées pour annuler un groupe donné de contraintes de requête. Par exemple, la requête suivante exclut les produits qui sont en liquidation ou dont le prix est inférieur à 10 :

```
$products = DB::table('products')
    ->whereNot(function ($query) {
        $query->where('clearance', true)
        ->orWhere('price', '<', 10);
    })
    ->get();
```

Des clauses Where additionnelles

```
$users = DB::table('users')
->whereBetween('votes', [1, 100])
->whereNotBetween('votes', [1, 100])
->whereBetweenColumns('weight', ['minimum_allowed_weight',
'maximum_allowed_weight'])
->whereNotBetweenColumns('weight', ['minimum_allowed_weight',
'maximum_allowed_weight'])
->whereIn('id', [1, 2, 3]) ->whereNotIn('id', [1, 2, 3])
->whereNull('updated_at')
->whereNotNull('updated_at')
->whereDate('created_at', '2016-12-31')
->whereDay('created_at', '31')
->get();
```

Ordre, regroupement, limite et offset



OrderBy

La méthode **orderBy** permet de trier les résultats de la requête par une colonne donnée. Le premier argument accepté par la méthode **orderBy** doit être la colonne sur laquelle vous souhaitez effectuer le tri, tandis que le second argument détermine la direction du tri et peut être **asc** ou **desc** :

```
$users = DB::table('users')  
    ->orderBy('name', 'desc')  
    ->get();
```

latest et oldest

Les méthodes **latest** et **oldest** vous permettent de classer facilement les résultats par **date**. Par défaut, le résultat sera ordonné par la colonne **created_at** de la table. Vous pouvez également transmettre le nom de la colonne sur laquelle vous souhaitez effectuer le tri :

```
$user = DB::table('users')  
    ->latest()  
    ->first();
```

inRandomOrder

La méthode **inRandomOrder** peut être utilisée pour trier les résultats de la requête de manière aléatoire. Par exemple, vous pouvez utiliser cette méthode pour extraire un utilisateur aléatoire :

```
$randomUser = DB::table('users')  
    ->inRandomOrder()  
    ->first();
```


reorder

La méthode `reorder` supprime toutes les clauses "order by" qui ont été précédemment appliquées à la requête :

```
$query = DB::table('users')->orderBy('name');  
$unorderedUsers = $query->reorder()->get();
```

groupBy et having

Comme vous pouvez vous y attendre, les méthodes **groupBy** et **having** peuvent être utilisées pour regrouper les résultats de la requête. La signature de la méthode **having** est similaire à celle de la méthode **where** :

```
$users = DB::table('users')  
    ->groupBy('account_id')  
    ->having('account_id', '>', 100)  
    ->get();
```

skip et take

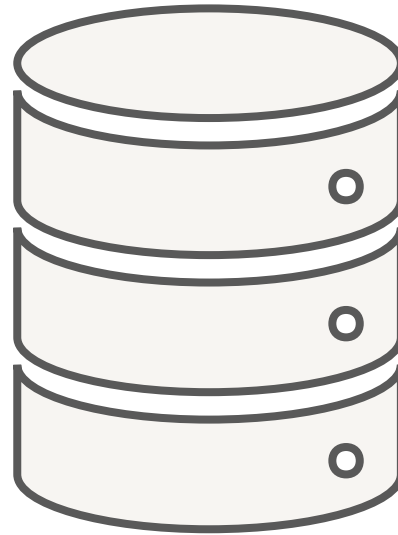
Vous pouvez utiliser les méthodes **skip** et **take** pour limiter le nombre de résultats renvoyés par la requête ou pour ignorer un nombre donné de résultats dans la requête :

```
$users = DB::table('users')->skip(10)->take(5)->get();
```

Vous pouvez également utiliser les méthodes **limit** et **offset**. Ces méthodes sont fonctionnellement équivalentes aux méthodes **take** et **skip**, respectivement :

```
$users = DB::table('users')->offset(10)->limit(5)->get();
```

INSERT



INSERT

Le générateur de requêtes fournit également la méthode **insert** qui peut être utilisée pour insérer des enregistrements dans la table de la base de données. La méthode d'insertion accepte un tableau de noms et de valeurs de colonnes :

```
DB::table('users')->insert([  
    'email' => 'kayla@example.com',  
    'votes' => 0  
]);
```

INSERT

Vous pouvez insérer plusieurs enregistrements à la fois en passant un tableau de tableaux. Chaque tableau représente un enregistrement qui doit être inséré dans la table :

```
DB::table('users')->insert([  
    ['email' => 'picard@example.com', 'votes' => 0],  
    ['email' => 'janeway@example.com', 'votes' => 0],  
]);
```

INSERT

La méthode **insertUsing** va insérer de nouveaux enregistrements dans la table tout en utilisant une sous-requête pour déterminer les données à insérer :

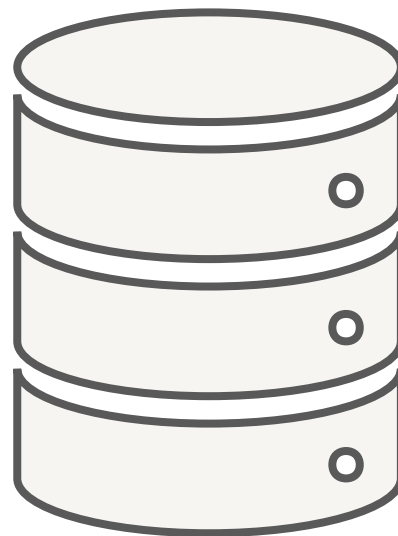
```
DB::table('pruned_users')->insertUsing([
    'id', 'name', 'email', 'email_verified_at'
], DB::table('users')->select(
    'id', 'name', 'email', 'email_verified_at'
)->where('updated_at', '<=', now()->subMonth()));
```

INSERT

La méthode **upsert** insère les enregistrements qui n'existent pas et met à jour les enregistrements qui existent déjà avec les nouvelles valeurs que vous pouvez spécifier.

```
DB::table('flights')->upsert(  
  [  
    ['departure' => 'Oakland', 'destination' => 'San Diego', 'price' => 99],  
    ['departure' => 'Chicago', 'destination' => 'New York', 'price' => 150]  
  ],  
  ['departure', 'destination'],  
  ['price']  
);
```


UPDATE



UPDATE

La méthode **update**, comme la méthode insert, accepte un tableau de paires de colonnes et de valeurs indiquant les colonnes à mettre à jour. La méthode de mise à jour renvoie le nombre de lignes affectées. Vous pouvez contraindre la requête de mise à jour à l'aide de clauses **where** :

```
$affected = DB::table('users')  
    ->where('id', 1)  
    ->update(['votes' => 1]);
```

UPDATE

Il peut arriver que vous souhaitiez mettre à jour un enregistrement existant dans la base de données ou le créer s'il n'existe aucun enregistrement correspondant. Dans ce scénario, la méthode **updateOrCreate** peut être utilisée.

```
DB::table('users')
    ->updateOrCreate(
        ['email' => 'john@example.com', 'name' => 'John'],
        ['votes' => '2']
    );
```

UPDATE

Le générateur de requêtes fournit également des méthodes pratiques pour incrémenter ou décrémenter la valeur d'une colonne donnée.

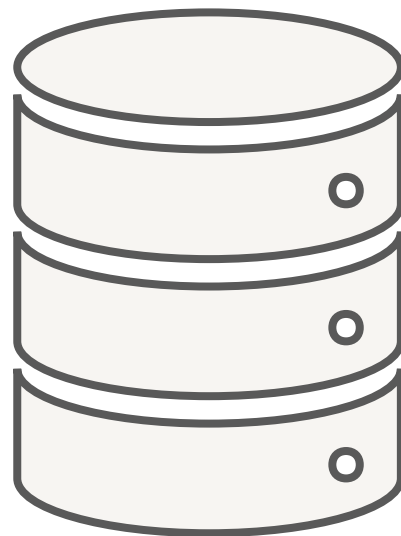
```
DB::table('users')->increment('votes');
```

```
DB::table('users')->increment('votes', 5);
```

```
DB::table('users')->decrement('votes');
```

```
DB::table('users')->decrement('votes', 5);
```

DELETE



DELETE

La méthode **delete** du constructeur de requêtes peut être utilisée pour supprimer des enregistrements de la table. La méthode **delete** renvoie le nombre de lignes affectées. Vous pouvez limiter les instructions de suppression en ajoutant des clauses "**where**" avant d'appeler la méthode **delete** :

```
$deleted = DB::table('users')->delete();
```

```
$deleted = DB::table('users')->where('votes', '>', 100)->delete();
```

DELETE

Si vous souhaitez tronquer une table entière, ce qui supprimera tous les enregistrements de la table et remettra l'ID auto-incrémenté à zéro, vous pouvez utiliser la méthode **truncate** :

```
DB::table('users')->truncate();
```